

CS305: Computer Networking Written Assignment1

12310520 芮煜涵

Q1

Application layer:

provide network services directly to the user, enabling applications to communicate over the network(eg.HTTP)

Transport layer:

providing end-to-end communication and exchange between devices(eg.TCP)

Network layer:

handle the routing and forwarding of data packets across the network, determining the best path(eg.IP)

Link layer:

handle physical transmission of data frames between two directly connected nodes(eg.Ethernet)

Physical layer:

define the hardware elements involved in the transmission and reception of raw data bits over physical media(eg.optical fiber)

Q2

(a)

Circuit-switching:

a communication method in which a dedicated communication path is established between two endpoints for the duration of the communication session, which is used in traditional telephone networks. The path is reserved during the communication.

Packet-switching:

a communication method where data is divided into small packets, which are transmitted independently. Packets may take different routes to the destination and are reassembled at the receiving end.

Difference:

Circuit-switching needs a dedicated path during the entire communication, even if there is no data to be transmitted, which is inefficient. However, packet-switching is more flexible and efficient, for it dynamically uses available network resources and doesn't require dedicated paths.

(b)

Client-server:

a network architecture where the clients request resources or services, while the server provides these resources and services, typically in a centralized manner.

Peer-to-peer:

a decentralized network architecture where every node is equal, or say acting as both client and server, provides resources or services without a central server.

Difference:

Client-server needs a centralized server to respond to clients' requests, while nodes in peer-to-peer can directly communicate with each other.

Q3

(a)

TCP is suitable for applications that require reliable communication without loss. Examples: web browsing, email, file transfer.

(b)

UDP is suitable for applications that require fast transmission and can tolerate some level of data loss, where speed matters more than reliability. Examples: online gaming, streaming, DNS.

Q4

(a)

$$\text{delay} = \text{transmission delay} + \text{propagation delay} = \frac{L \times K}{R} + K \times d = \frac{LK}{R} + Kd$$

(b)

$$\text{delay} = \text{setup time} + \text{transmission delay} + \text{propagation delay} = \tau + \frac{L}{R/F} + Kd = \tau + \frac{LF}{R} + Kd$$

(c)

$$T_{trans} : \text{transmission delay for each packet} = \frac{L}{R}$$

T_{proc} : nodal processing delay

T_{prop} : propagation delay between two nodes

$$\text{delay for packet1} = 2T_{trans} + T_{proc} + 2T_{prop} = \frac{2L}{R} + 5 \times 10^{-6} + 2 \times 10 \times 10^{-6} = \frac{2 \times 1000}{20 \times 10^6} + 25 \times 10^{-6} = 10^{-4} + 2.5 \times 10^{-5} = 1.25 \times 10^{-4}$$

$$\text{delay for two packets} = \text{delay for packet1} + T_{trans} = 1.25 \times 10^{-4} + \frac{1000}{20 \times 10^6} = 1.75 \times 10^{-4} = 175 \mu s$$

Q5

(a)

$$D_{trans} = \frac{10 \text{ Mbits}}{10 \text{ Mbps}} = 1 s$$

Every second one packet arrives, which means that once a packet arrives, it can be transmitted immediately without queuing.

Therefore, the average queue delay is 0s.

(b)

Every K second K packets arrive simultaneously, the first one of the K packets queues for 0s, while the last one queue for $(K - 1)s$.

Therefore, the average queue delay is $\frac{K-1}{2} s$.

(c)

$$\text{traffic intensity in (a): } \rho = \frac{L\lambda}{R} = \frac{10 \times 1}{10} = 1$$

$$\text{traffic intensity in (b): } \rho = \frac{L\lambda}{R} = \frac{10 \times K/K}{10} = 1$$

Insights: even though these two scenarios have the same traffic intensity, they may have different average queue delay. Traffic intensity alone cannot evaluate the queuing performance in a comprehensive manner.

Q6

(a)

HTTP runs on TCP.

TCP provides a reliable, connection-oriented data transfer service. It ensures that all HTTP messages are delivered in order and experience no data loss. However, UDP doesn't guarantee reliability and the order of delivery, though it can build fast connection, it is unsuitable for HTTP.

(b)

HTTP request message, for it begins with a request line `GET...`.

(c)

Persistent connection, for the line `Connection:keep-alive`.

(d)

Corresponding entry:

```
1 | Host: gaia.cs.umass.edu
2 | ID=1150
```

Header line included:

```
1 | Cookie: ID=1150
```

(e)

Status line:

```
1 | HTTP/1.1 200 OK\r\n
```

Entity body includes:

the HTML content of the file `/cs453/index.html` returned by the server.

Q7

(a)

$$\Delta = \frac{1}{20} = 0.05s$$

$$\text{average response time} = \text{internet delay} + \text{average access time} = 2 + \frac{\Delta}{1-\Delta\beta} = 2 + \frac{0.05}{1-0.05 \times 10} = 2.1s$$

(b)

hit rate : x

$$\text{average access delay without hit} = \frac{\Delta}{1-\Delta\beta'} = \frac{0.05}{1-0.05 \times 10(1-x)} = \frac{0.1}{1+x} s$$

$$\text{average access delay} = (1-x)\left(2 + \frac{0.1}{1+x}\right)$$

$$\text{let } (1-x)\left(2 + \frac{0.1}{1+x}\right) < 1$$

$$\text{then } x > 0.516$$

Therefore, the hit rate must be over 51.6%.

Q8

(a)

$$\text{transmission time for one file} = \frac{L}{R}$$

DNS query only once, and UDP: RTT_0

$$\text{for every object: } 2RTT_1 + T_{trans} = 2RTT_1 + \frac{L}{R}$$

$$\text{Therefore, for the whole process, time needed} = RTT_0 + 13\left(2RTT_1 + \frac{L}{R}\right) = RTT_0 + 26RTT_1 + \frac{13L}{R}$$

(b)

For 4 parallel connections, we need to first fetch HTTP basic file; after that we can fetch 12 objects with 4 connections.

$$\text{Therefore, for the whole process, time needed} = RTT_0 + 2RTT_1 + \frac{L}{R} + 12/4 \times \left(2RTT_1 + \frac{L}{R}\right) = RTT_0 + 8RTT_1 + \frac{4L}{R}$$

(c)

For persistent HTTP, we only need $3RTT_1$,

$$\text{Therefore, for the whole process, time needed} = RTT_0 + 3RTT_1 + \frac{13L}{R}$$

Q9

(a)

When $\frac{u_s}{N} \leq d_{min}$:

The server divides its upload rate equally among all peers, sending at $\frac{u_s}{N}$ to each.

$$\text{Distribution time: } T = \frac{NF}{u_s}$$

(b)

When $\frac{u_s}{N} \geq d_{min}$:

Each peer receives at its full download rate d_{min} .

$$\text{Distribution time: } T = \frac{F}{d_{min}}$$

(c)

In general, any distribution scheme must satisfy two lower bounds:

- The server must send a total of NF bits at a maximum rate of u_s , so the time is at least $\frac{NF}{u_s}$.

- The slowest peer has a maximum download rate of d_{min} ,
so the time is at least $\frac{F}{d_{min}}$.

Since (a) and (b) each provide a feasible scheme achieving these bounds,
the **minimum distribution time** is:

$$T_{min} = \max \left\{ \frac{NF}{u_s}, \frac{F}{d_{min}} \right\} \quad (1)$$

Q10

A hierarchical DNS system is used instead of a single centralized server because it improves scalability, reliability, and efficiency.

A huge centralized DNS would be a single point of failure, cause heavy traffic congestion, and create long delays.

The hierarchical design (root → TLD → authoritative servers) distributes the load and allows faster, local caching of domain information.